

Learning System 4 & 5: Random Forest, Generalization and Regularization

DT8008 VT26, Halmstad University

Binay Kumar Sah, binsah25@student.hh.se, Master's in Embedded and Intelligence System
Anton Bauer, antibau25@student.hh.se, TAEIS 25

March 2026

Introduction

This lab goes over the concepts of ensemble learning and regularization to help the models generalize. We implemented a random forest classifier in part A, where we use the combined decision of multiple individually trained trees - gaining a more robust/generalized end model. For part B we implemented regularized linear regression to predict water flow from a dam, which provided insight into the effects of bias and variance. Lastly, for part C we created a regularized logistic regression to classify faulty microchips, where the non-linear structure of the data showed the importance of appropriate feature mapping.

Lab 4: Random Forest Classifier

Objective

In Part A, we were tasked with implementing both a simple decision tree, and a random forest classifier, to classify if a student was admitted given the scores of two tests.

Key Concepts

- **Decision Tree:** Is a supervised model that makes predictions through splitting the data into subsections that minimizes the impurity. It is a hierarchical process and allows for hyperparameters such as depth to help with generalization.
- **Random Forest:** Is an ensemble learning method that utilizes multiple decision trees, all trained on different data through the use of randomness - hence the name random forest. It combines the results of the multiple trees by taking the mode, or in other words, the most voted for classification for each prediction (majority vote).
- **Random Subspace Method:** Is the algorithm responsible for randomly selecting a subset of features $n_features$ and a subset of data points $n_samples$. It is an important aspect of ensemble learning as it allows for trees to view the data through different lenses, which ensures diversity and increases generalization.

Algorithm (Random Forest)

Algorithm 1 Random Forest using Random Subspace Method

- 1: **Step 1: Data Subsampling**
 - 2: Use the random subspace method to obtain a subset of the data, where parameters can specify the specific number of features and samples to use.
 - 3: **Step 2: Tree Construction**
 - 4: Create a decision tree with the subset from previous step.
 - 5: **Step 3: Ensemble Building**
 - 6: Repeat the steps above until the number of trees desired is achieved.
 - 7: **Step 4: Parallel Prediction**
 - 8: To predict, pass the new data samples to each tree model and store all their predictions.
 - 9: **Step 5: Aggregation**
 - 10: Finalize the prediction of the "forest" by taking the majority vote among the trees's predictions.
-

Observation and Results:

We observed that a single simple decision tree achieved a training accuracy of 100% whilst the random forest got an accuracy of 98%. The lower accuracy of the forest entails that the ensemble of multiple trees helped generalization, as a training accuracy of 100% is highly likely overfitting and not capturing the underlying pattern. If we would have implemented the optional part (with 10-fold-cross-validation) it would likely have become apparent that the random forest, with its lower training accuracy, would outperform the single tree due to better generalization.

Lab 5 - Part A: Regularized Linear Regression

Objective

In Part B, we implemented regularized linear regression to predict the water flowing out of a dam given the change of water level in a reservoir. With heavy focus on the impacts of bias vs variance.

Key Concepts

- **Regularization:** Is the technique used to combat overfitting by adding a penalty term to the cost function. The penalty term can take on different shapes depending on the desired effects. The one used in the lab is called Ridge Regression and looks like:

$$\lambda \left(\sum_{j=1}^d \theta_j^2 \right)$$

Resulting in large feature coefficients (θ) being heavily penalized, forcing the model to use smaller and more distributed coefficients that effectively help the model generalize.

- **Bias vs. Variance:** Bias is the ability of the model to accurately represent/capture the underlying pattern of the data. High bias equates to a model that is too simple (underfitting) and fails to capture the pattern. Variance instead regards the model's sensitivity to the specific training data used. A high variance entails a model that has over-learned the training set, essentially capturing the noise instead of a generalized pattern (overfitting). The problem is that a model perfectly capturing the training data (low bias) often results in high variance, as it ends up capturing all the noise in the training data; it then follows that it would not perform well on unseen data. Therefore, it is important to strike a balance between bias and variance to ensure the model generalizes well.

Algorithm

Algorithm 2 Regularized Linear Regression and Polynomial Modeling

- 1: **Step 1: Simple Model**
 - 2: Initialize parameters for linear regression model: initial thetas, lambda λ , training data (X, y) .
 - 3: **Step 2: Cost and Training**
 - 4: Compute the regularized cost function (adding regularization term to cost function) and gradient.
 - 5: Train the model (finding optimal θ s) using optimization function (*scipy's minimize*).
 - 6: **Step 3: Visualize Learning**
 - 7: Iteratively train the model on increasing subsets of the training data.
 - 8: Then compute and plot the training and validation errors to see if the model suffers from high bias or high variance. {Generates Learning Curves}
 - 9: **Step 4: Polynomial Model**
 - 10: To address the high bias (underfitting) of the simple model, utilize a mapping of the original features into higher-degree polynomial terms to get a more complex feature space.
 - 11: **Step 5: Feature Scaling**
 - 12: Apply feature scaling (standardization) to these newly created polynomial features to ensure stable and efficient optimization.
 - 13: **Step 6: Regularized Polynomial Model**
 - 14: Perform 10-fold cross-validation on the combined training and validation sets across a predefined range of λ values to find the most suitable/optimal one.
 - 15: **Step 7: Final Evaluation**
 - 16: Train the final polynomial model using the found optimal λ on the entire training dataset, and evaluate its performance against the unseen test set.
-

Observation and Results

We initially observed that a simple linear regression model resulted in high errors for both the training and validation sets. As seen by the fitted line in Fig 1a and the error rates in Fig 1b, a straight line cannot capture the curved nature of the dam's water flow. Consequently, the learning curve stays at a high error rate and shows that the simple model underfitted (high bias).

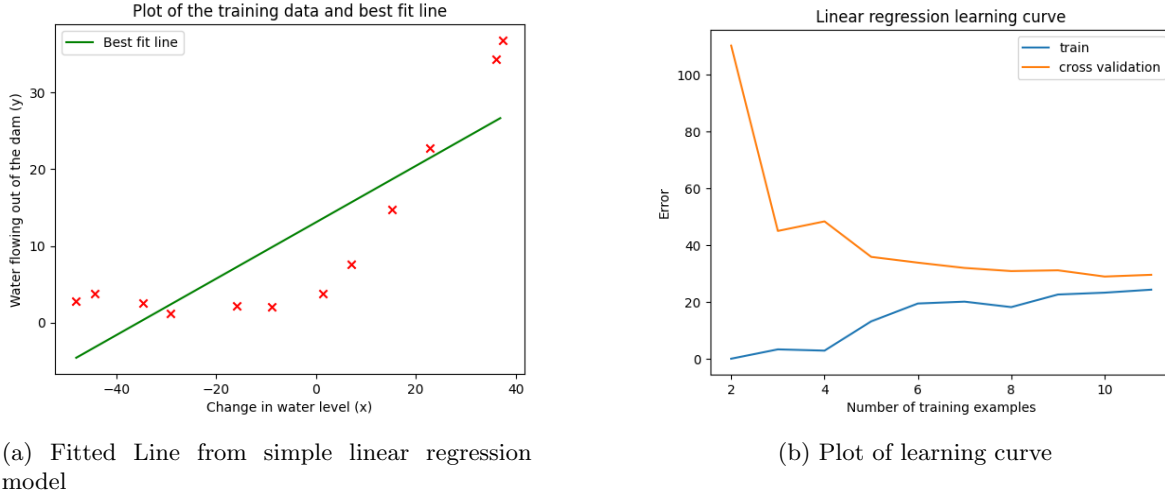


Figure 1: Plots regarding simple linear regression model

To solve this lack of complexity a simple line is not enough, and as such, we mapped the features into an 8th-degree polynomial. As seen by the learning curve in Fig 2, this approach reduced the training error to nearly zero. However, a significant sporadic gap between the training and validation errors

remained. This suggests that the unregularized polynomial model ($\lambda = 0$) ended up memorizing the training data and its noise, resulting in overfitting (high variance) and poor generalization to unseen data - as seen by the left plot in Fig 2 following the data points exactly. Regularization improves the generalization of the model as seen in Fig 3, as the curve no longer follows the data to an exact and consequently achieves much better generalization (lower validation error), but at the cost of slightly higher training error (higher bias).

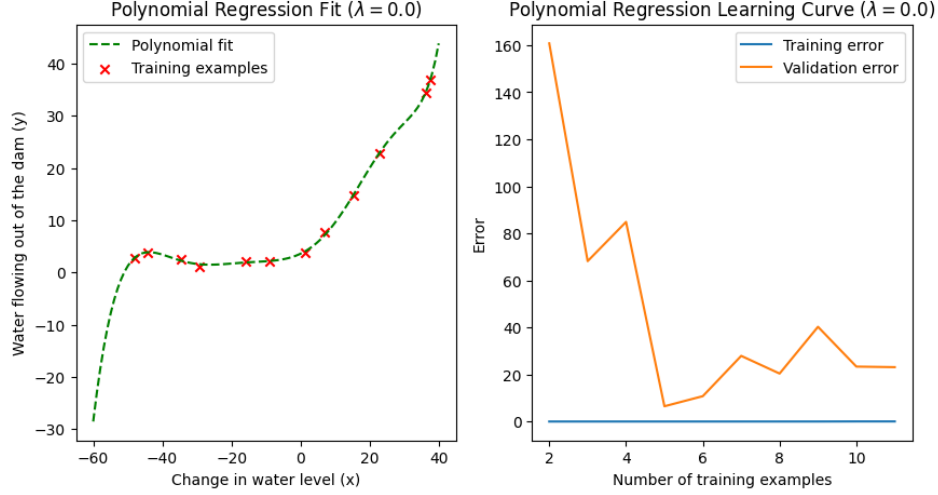


Figure 2: Plots for polynomial model with $\lambda = 0$, showcasing the fitted curve (left) and the learning curve (right).

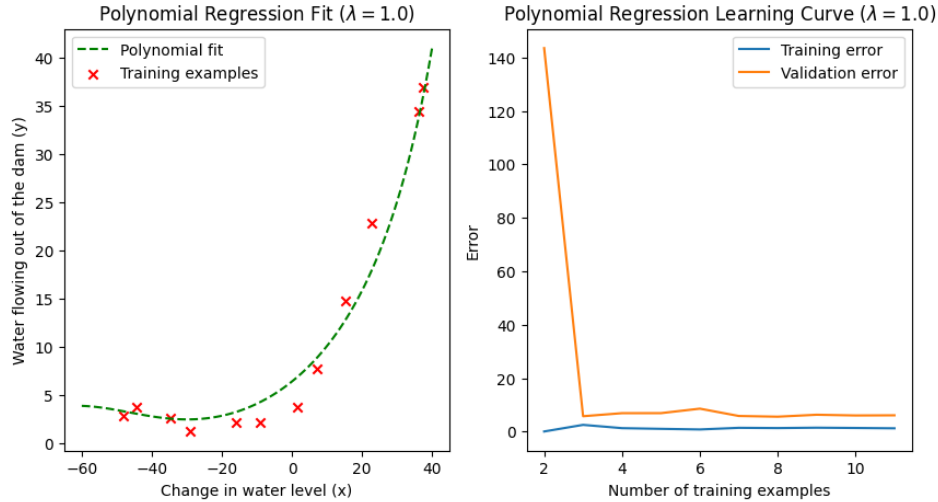


Figure 3: Plots for polynomial model with $\lambda = 1$, showcasing the fitted curve (left) and the learning curve (right).

This leads into the final part of our observations where we want to find the best lambda; the one that strikes the best balance between bias and variance. We find it by applying 10-fold cross-validation, giving an optimal regularization penalty of $\lambda = 3$ through observing the minimum training and test error in Fig 4. As can be seen in Fig 5, this final model achieved a much lower test error, proving that the simple model was too rigid; the unregularized polynomial was too chaotic; the regularized polynomial model was the perfect balance thanks to using the optimal $\lambda = 3$.

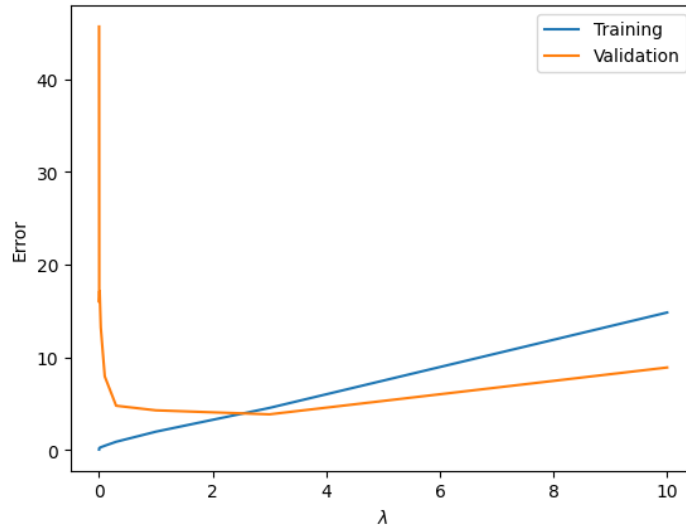


Figure 4: Plots the validation and training error for the regularized polynomial model for a range of regularization terms (λ), showing the optimal value of λ to be around 3.

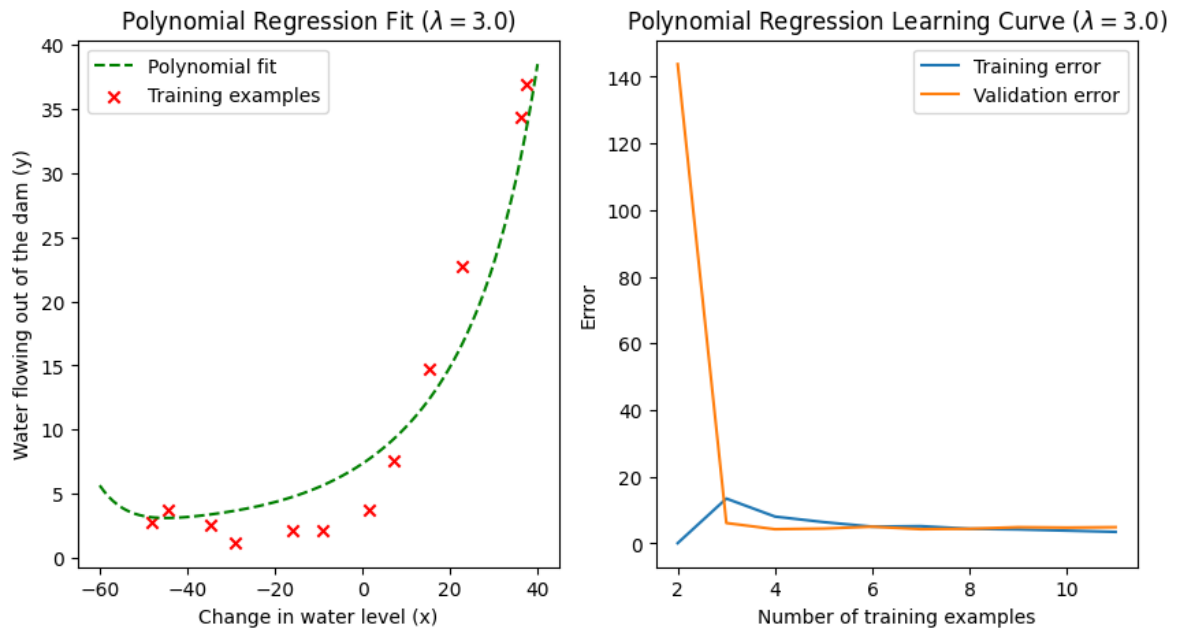


Figure 5: Plots for polynomial model with $\lambda = 3$ (optimal λ), showcasing the fitted curve (left) and the learning curve (right).

Lab 5 - Part B: Regularized Logistic Regression

Objective

In this section we implemented the Regularized Logistic Regression to build a classification model that predicts whether microchips pass quality assurance based on two test results.

Key Concepts

- **Feature Mapping (Polynomial Expansion):** Since the data was not linearly separable, so to create curved boundaries, Feature Mapping was used. This involves taking our two original features (x_1, x_2) and mathematically creating new ones (like x_1^2, x_1x_2, x_2^2 , up to the 6th power). This moves our data into a "higher-dimensional space" where a complex boundary is possible.
- **Bias - Variance Trade-off:** We visualized the balances at different value of λ . When it was too large, the model was too stiff and misses the pattern while it was too wiggly when it was too small.

Algorithm

Algorithm 3 Regularized Logistic Regression with Feature Mapping

- 1: **Step 1: Load And Visualize the data**
 - 2: Load the microchip test results and visualize the dataset using a scatter plot to observe the non-linear distribution of the two classes (Accepted vs. Rejected).
 - 3: **Step 2: Feature Mapping**
 - 4: To handle non-linearly separable data, map the original two features into all polynomial terms up to the sixth power to create a higher-dimensional feature space (28 features).
 - 5: **Step 3: Regularized Cost Function**
 - 6: Define the regularized logistic regression cost function and gradient, incorporating the parameter λ to penalize large theta values.
 - 7: **Step 3: Define Regularized Cost Function**
 - 8: $E(\theta) = -\frac{1}{n} \sum_{i=1}^n [y \log(h_{\theta}(X)) + (1 - y) \log(1 - h_{\theta}(X))] + \frac{\lambda}{2n} \sum_{j=1}^n \theta_j^2$
 - 9: **Step 4: Model Training**
 - 10: Train the model (finding optimal θ) using optimization function (scipy's minimize).
 - 11: **Step 5: Visualizing the Decision Boundary**
 - 12: Plot the non-linear decision boundary over the training data to visually how the model separates the classes.
 - 13: **Step 6: Evaluating Regularization (λ)**
 - 14: Observe the impact of different λ values (e.g., 0, 0.5, 1, 10, 100) on the decision boundary to identify cases of overfitting (high variance) or underfitting (high bias).
-

Observation and Results

We implemented our model and observe our decision boundaries at different values of λ (0, 0.01, 0.5, 1, 10 and 100) and the outcomes are shown in figure 6.

We can clearly see the relationship between the regularization parameter (λ) and the model's decision boundary. When $\lambda = 0$, the model exhibited High Variance (Overfitting), creating a highly complex boundary that captured noise in the training set as we can see in figure 6a whereas when tested at $\lambda = 10$, further increased to 100, the model suffered from High Bias (Underfitting), decision boundary shrinks resulting in a boundary that was too simple to represent the data accurately, fig 6f.

However, the most balanced performance was observed between $\lambda = 0.5$ to $\lambda = 1$, where the boundary maintained a smooth, non-linear shape that followed the general data distribution without

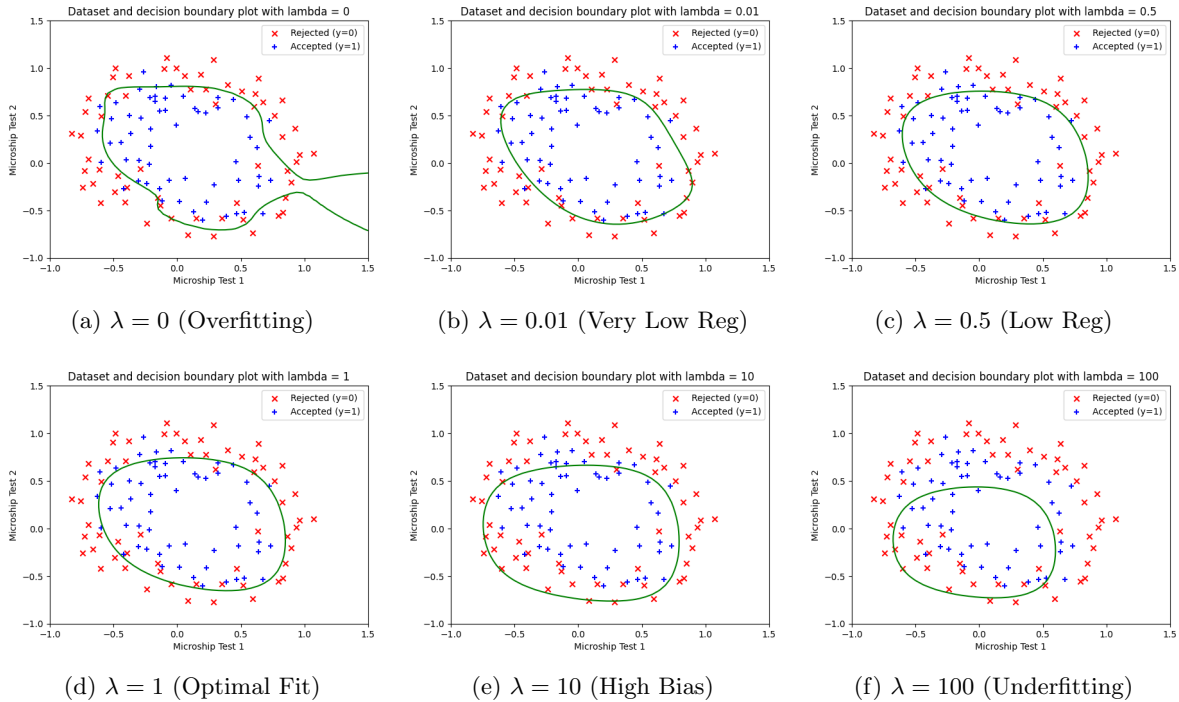


Figure 6: Effect of Regularization Parameter (λ) on the Decision Boundary. As λ increases, the model moves from high variance (complex/wiggly boundary) to high bias (simple/rigid boundary).

over-reacting to outliers. It balances the need to fit the data (Low Bias) with the need to stay simple (Low Variance) and hence this model will likely have the best performance on new, unseen microchips.

Conclusion

Lab 4 & 5 provided good insight into how and why generalization plays a large part in machine learning. In Part A, we got to look at how ensemble learning can help combat overfitting through the random subspace method combined with majority voting, which we did firsthand by moving from a single decision tree to implementing a random forest classifier. Part B and C gave us insight into bias and variance, where we realized that mapping to polynomial features greatly helps with underfitting. But, also how this introduces the contrary problem - with the model now being prone to overfitting instead. A big takeaway following this was the importance of the penalty term λ to combat overfitting, where taking time to find the most appropriate penalty term massively improves the generalized performance of our model. All-in-all this lab provided the tools to help us "tame" our models and force them to actually learn the underlying pattern, and knowledge to avoid false perceptions; such as high training score translating into good overall performance.